

Codetopia Mentorship Program - 2026

Git & GitHub



Session 1:

Foundation



What we will cover today

- What is version control and why it matters
- The 3 types of version control systems
- The 3 Git states: Working Directory, Staging Area, Repository
- Installing Git on your machine
- Configuring your identity with git config
- CLI vs GUI — which to use and when
- Session Project: Setup Verification Report



Community

Version Control System



What is Version Control System

Three types of version control

Type	Description
Local VCS	Saves versions only on your computer (e.g. numbered folders)
Centralised VCS	One shared server holds all versions (e.g. SVN)
Distributed VCS	Every user has the full history, this is Git



What is Version Control System

Why distributed wins

- Work offline — full history is on your machine
- No single point of failure
- Branching and merging are fast and cheap

Version Control - Git



What is Git?

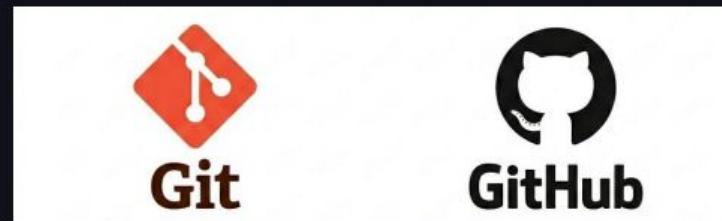
Git at glance:

- Git is a free, open-source distributed version control system created by Linus Torvalds in 2005.
- Originally built to manage the Linux kernel source code, one of the largest open-source projects in the world.
- Today it is the most widely used version control system: over 90 % of developers use Git daily.

What is Git?

Git ≠ Github

- Git is the tool that runs on your machine and tracks changes locally.
- GitHub (and GitLab, Bitbucket, etc.) are hosting platforms that store Git repositories online and add collaboration features like pull requests and issues.



Git States

Every file in Git lives in one of three states

- Working Directory — where you edit files on your computer
- Staging Area (Index) — a preparation zone for your next commit
- Repository (.git folder) — the permanent record of all commits

The Flow

Edit file → `git add` → `git commit`

Installing Git

Installation by OS

Operating System	Command/Method
macOS	<code>brew install git</code> OR download from git-scm.com
Windows	Download Git for Windows installer at git-scm.com
Ubuntu / Debian	<code>sudo apt-get install git</code>
Fedora	<code>sudo dnf install git</code>

Installing Git

Verify Install:

```
git --version
```

Expected Outcome

```
git version 2.x.x
```

Configuring Your Identity

Set your name and email globally

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

Verify your config

```
git config --list
```

Why this matters

- Every commit you make will be signed with this identity
- Collaborators and employers will see this on GitHub
- Use your real name, this is your professional portfolio



Community

CLI vs GUI

CLI vs GUI

Which Should you use?

CLI (Command Line)	GUI (e.g. GitHub Desktop, GitKraken)
Works everywhere, always available	Visual and beginner-friendly
Full control over every option	Hides some advanced features
Required for servers and CI/CD	Great for reviewing diffs visually
What employers expect you to know	Useful as a companion, not a replacement

CLI vs GUI

Our approach in this course:

- We learn CLI first — so you understand what's actually happening
- You can use a GUI alongside, but CLI is the foundation



Session Project

Git Installation & Setup

Session Project

Your deliverables

1. Run `git --version` and take a screenshot
2. Set your name and email with `git config --global`, verify with `git config --list` and take a screenshot.
3. Create a folder called `session-1` and type `git init`
4. Create `setup-notes.md` — write the 3 Git states in your own words
5. Commit the file with message: `docs: add setup verification notes`
6. Write 2 sentences comparing CLI vs GUI — when is each more useful?

References & Further Reading

Resources

Text Resource

- The Odin Project:
Prerequisites

<https://www.theodinproject.com/lessons/foundations-prerequisites>

- Video Resource FreeCodeCamp Git & GitHub Crash Course
(first 10 mins)

<https://www.youtube.com/watch?v=RG0j5yH7evk>